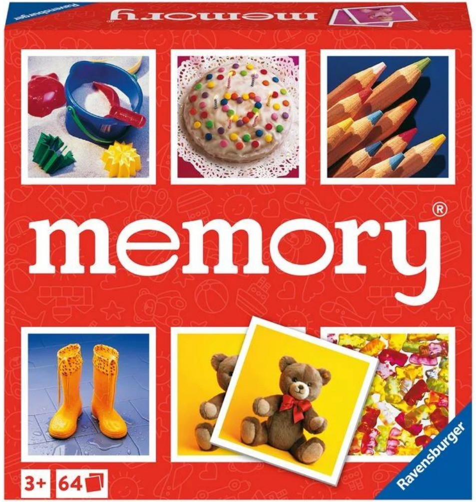
GRU-ODE

Gabriel Brown & Nino Sabella

SLIDE 1: THREE ELEMENTS

TIME

MEMORY



INFO

SLIDE 1: three elements **OF A GRU MODEL**

1. **TIME**
2. **MEMORY**
3. **INFO**

GRU = Gated Recurrent Unit = Type of Recurrent Neural Network

GRUs work with **discrete** time intervals
- Ex: 1.0, 2.0, 3.0
- Ex: Monday, Tuesday, Wednesday

THE WAY IT WORKS - these models keep learning
- Learning process:
  - Given new input (knowledge), keep some info and lose some in the memory - the model decides what's relevant
- Apply memory learned to a prediction
- Next time interval brings new info so repeat

SLIDE 1: three elements of a GRU model'S PREDICTION PROCESS

1. **DISCRETE** TIME
2. **SELECTIVE** MEMORY
3. **NEW** INFO

GRU Memory Update:

- Ht = GRUCell(Ht-1, Xt)
  - Ht: Updated Memory
  - Ht-1: Memory Before New Info Came In
  - t = Current Discrete Time Moment
  - t-1 = Most Recent Old Discrete Time Moment
  - Xt: New Information AOTMT (as of this moment in time)

GRU Prediction:

- Yt = W · Ht + b
  - W, b: weights of linear output layer (to help guide prediction)
  - Yt: the prediction of the GRU model

SLIDE 1: three elements of a GRU model's **prediction process IN AN EXAMPLE**

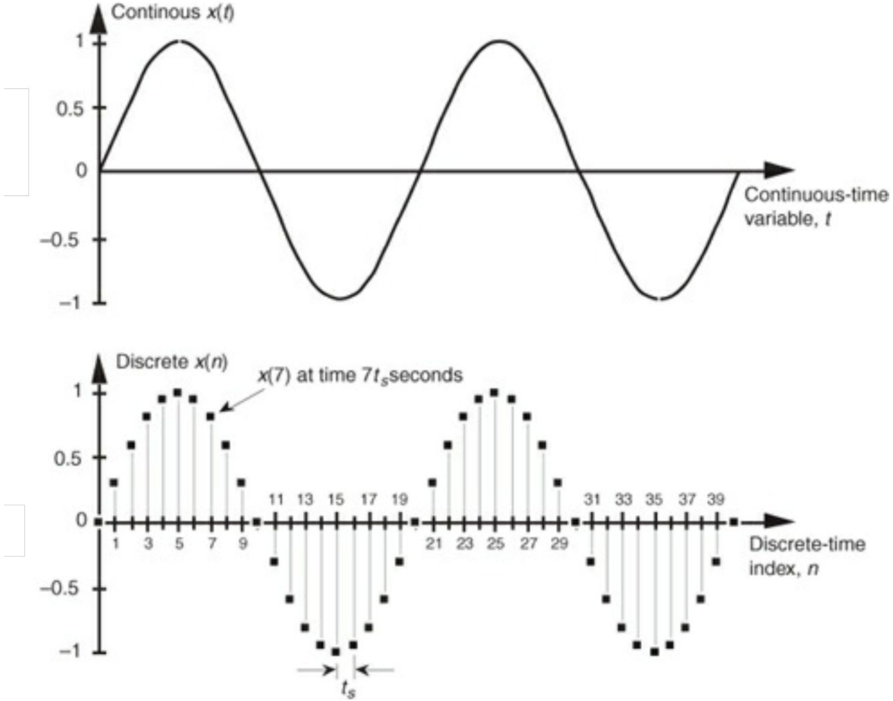
1. **DISCRETE** TIME
2. **SELECTIVE** MEMORY
3. **NEW** INFO

TRACKING CRISTIANO RONALDO'S ENERGY LEVEL IN TRAINING:

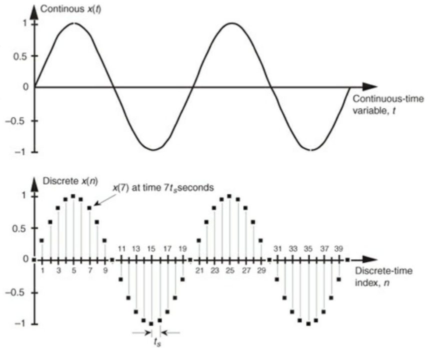
Time: 1 Hour Intervals	**Input: Energy Sensor Result**	**GRU Action**	**Memory State**
After 1 hour (t=1)	0.95 (1.00 is max energy)	Updates memory to "fresh"	Fresh (as vector of numbers)
After 2 hours	0.9	Updates to "pretty fresh"	Pretty fresh
After 3 hours	0.6	Updates to "tired"	Tired

SLIDE 2: GRU’S KRYPTONITE

SLIDE 2: GRU’s kryptonite – CONTINUOUS TIME



SLIDE 2: GRU’s kryptonite – CONTINUOUS TIME



TRACKING CRISTIANO RONALDO’S ENERGY LEVEL IN TRAINING:

Time: 1 Hour Intervals	**Input Se**	**ction**	**Memory State**
After 1 hour (t=1)	0		Fresh (as vector of numbers)
After 2 hours		n”	Pretty fresh
After 3 hours	0.6	s to	Tired

WHAT HAPPENS IF COACH DOESN’T CHECK EVERY HOUR? What if 30 min, then 10 min, then 60 min?

SLIDE 2: SOLVING GRU’s kryptonite – continuous time

Queue in ODE: Ordinary Differential Equation

- ODE is like having an autonomous fitness model that **tracks Cristiano’s energy fluctuations even in moments when we aren’t entering his energy sensor results**

SIMULATION OF DATA BETWEEN OBSERVATIONS

Combining GRU and ODE: *GRU-ODE*
- GRU takes in updates about real new data
- ODE simulates gaps between real updates so model’s memory isn’t outdated

SLIDE 3: INSIDE THE ODE PROCESS

ODE Understanding of Smooth Data Changes:

- $dH(t) / dt = f\theta(H(t))$
  - Ht: the GRU's memory at time t (also called hidden state)
  - $dH(t) / dt$: how the hidden state (model memory) changes with time
    - **This is the speed and direction at which data SMOOTHLY changes, which memory must simulate**
      - assumes smooth changes
  - $f\theta$: a small neural network that **learns the speed** at which memory changes with time
  - θ: model weights learned during training

SLIDE 3: inside the ODE process – FILLING GAPS

Using Continuous Evolution to Move Memory Forward in Time:

- H(t2) = H(t1) + ∫ fθ(H(t)) dt
    - H(t2): the GRU's updated memory at time t2 (**updated without seeing new data**)
    - H(t1): GRU's memory (hidden state) at time t1
    - ∫ dt: Integration of small changes in values between times t1 and t2 (the changes simulated by ODE)
        - ODE magic happens in the fθ(H(t): function (small neural network) tells the rate and direction of change in values for the given time period.

SLIDE 4: Concrete Prediction & Use Examples

Cristiano's Scenario:

- GRU-ODE model could be used to predict:
    - Next energy level (what will his energy be in 15 minutes from now)
    - Fatigue risk (is he about to overdo it)
    - Recovery time (how long it takes him to get back to 1.00 energy)
- This information could be used for in-game decision making regarding substitutions and tactics

Areas of Use:

- GRU-ODE model is useful for:
    - Patient monitoring (heart, blood sugar, etc)
    - Disease prediction
    - Tracking animal movements or behavior
    - Predicting machine failures from irregular sensor logs

GRU-ODE is great for any case where data comes in at uneven intervals that need to be synchronized to enable effective pattern learning.

SLIDE 5: Weaknesses of GRU-ODE

- Computational cost
  - Training can be slow compared to normal GRU operations (ODE adds complexity)
- Harder to optimize
  - Gradient stability is an issue (exploding and vanishing when backpropagating)
- Assumes smoothness (doesn’t handle large jumps in values well)
- The data must be irregular for it to be worth it (evenly spaced data can be modeled with just GRU)